CVE-2025-26943 is a classic example of a **SQL injection vulnerability in a WordPress plugin**, in this case caused by the Easy Quotes plugin failing to properly handle user input before using it in a database query.

Below is a clear breakdown.

---

1. Why this vulnerability happens

SQL injection occurs when **untrusted input is directly inserted into an SQL query** without proper validation or parameterization.

In WordPress plugins like Easy Quotes, this typically happens because:

🔴 Unsanitized input is used in queries

Developers may do something like:

- Reading `$_GET`, `$_POST`, or `$_REQUEST` values
- Directly concatenating them into SQL strings

Example of the *problem pattern*:

$sql = "SELECT * FROM wp_quotes WHERE id = " . $_GET['id'];

🔴 Missing prepared statements

Instead of using WordPress-safe database APIs (like `$wpdb->prepare()`), raw string building is used.

🔴 Lack of input validation

Even basic checks like:

- ensuring integers are integers
- validating allowed values
- sanitizing strings

are missing or inconsistent.

🔴 Trusting user-controlled request data

WordPress plugins often assume admin panels or shortcodes are “safe”, but attackers can still manipulate requests directly.

---

2. How developers should fix it

To properly fix SQL injection vulnerabilities, developers should apply **defense in depth**:

✅ 1. Use prepared statements (MOST IMPORTANT)

WordPress provides `$wpdb->prepare()` specifically to prevent SQL injection.

✅ 2. Validate and sanitize input

- Use `intval()` for numeric IDs
- Use `sanitize_text_field()` for strings
- Use whitelists for enums/actions

✅ 3. Enforce capability checks

Ensure only authorized users can trigger sensitive queries:

current_user_can('manage_options')

✅ 4. Use proper WordPress APIs where possible

Instead of raw SQL, use:

- `get_posts()`
- `WP_Query`
- `wpdb` helper methods

✅ 5. Escape output (separate from SQL injection fix)

Use:

- `esc_html()`
- `esc_attr()`

---

3. Secure coding example

Here’s how the vulnerable query should be rewritten safely:

❌ Vulnerable version

global $wpdb;

$id = $_GET['id'];

```
$sql = "SELECT * FROM {$wpdb->prefix}quotes WHERE id = $id";

$result = $wpdb->get_results($sql);
```

✅ Secure version (correct fix)

```
global $wpdb;

// 1. Validate input
$id = isset($_GET['id']) ? intval($_GET['id']) : 0;

if ($id <= 0) {
    wp_die('Invalid ID');
}

// 2. Prepare statement safely
$sql = $wpdb->prepare(
    "SELECT * FROM {$wpdb->prefix}quotes WHERE id = %d",
    $id
);

// 3. Execute query
$result = $wpdb->get_results($sql);
```

🔐 Even better (with capability check)

```
if (!current_user_can('manage_options')) {
    wp_die('Unauthorized');
}

global $wpdb;

$id = isset($_GET['id']) ? intval($_GET['id']) : 0;

$sql = $wpdb->prepare(
    "SELECT * FROM {$wpdb->prefix}quotes WHERE id = %d",
    $id
);

$result = $wpdb->get_results($sql);
```

Key takeaway

This vulnerability happens because:

> **user input is treated as trusted SQL code instead of data**

And the fix is always:

> **use parameterized queries + validate input + restrict access**

If you want, I can also show:

- how attackers typically exploit SQL injection in WordPress plugins (high-level, non-payload)
- or how to audit a plugin for similar CVEs quickly